Speaker notes — Marek's slides (TP2, Genetic Algorithms / Image Approximation)

Your marked slides, in order:

1. **p7** — Exercise 2 intro ("Image Approximation Using Triangles")
2. **p8** — Exercise 2: Input / Output / Optimization + `score = 1/(1+k·error)`
3. **p10** — Fitness Function (MSE)
4. **p12** — Crossover (left half of "Crossover & Mutation")
5. **p13** — Genetic Algorithm Flow
6. **p15** — Design Choices & Justification: the **Fitness–MSE** and **Termination** rows
7. **p27** — Conclusions: the **Possible improvements** list

Key facts that anchor everything (memorise these):

- **Gene** = one translucent triangle: 3 vertices `(x,y)` + RGBA colour, every value normalised to `[0,1]`.
- **Individual** = a fixed-length list of triangles (fixed = a run parameter, e.g. 50). **Population** = many such lists.
- **Fitness** = `1 / (1 + k·error)` with `k = 1`; **error** = mean squared error between the rendered candidate and the target, per RGB channel, on values scaled to `[0,1]` → MSE ∈ `[0,1]`, 0 = identical.
- **Rendering**: pygame, white background, each triangle drawn on its own alpha layer and alpha-blended on top of the previous ones **in gene order** → draw order is part of the phenotype.
- **Experiment setup**: target = Argentina flag downscaled to 100 px (`flag_100.png`), 5 seeds (1–5), one parameter changed at a time from a baseline (Elite / one-point / gene / 50 triangles / pop 50 / additive / 1000 generations).

Slide p7 — Exercise 2 intro

Say (~25 s):

> "*The second exercise is image approximation. The goal is to reproduce a target image using a **fixed number of semi-transparent, uniformly-coloured triangles**. Each triangle has one flat RGBA colour — no gradients — but because they're translucent and overlap, a handful of them can build up smooth colour transitions. Here's the idea on Van Gogh's Starry Night: on the left the target, on the right an approximation evolved by the genetic algorithm. We're not drawing this by hand — the GA searches for the set of triangles whose rendering looks closest to the target.*"

Points to hit: - Fixed triangle count → fixed-length genome → simpler GA (no variable-length individuals). - "Uniformly coloured" = each triangle is a single flat colour; overlap + transparency is what creates detail. - *Starry Night* is just an illustration; the measured experiments use a simpler target (the Argentina flag).

Likely questions: - *Why triangles and not circles/curves/pixels?* Triangles are the simplest 2D primitive that can tile any shape; 3 vertices + colour is a small, uniform parameter block that's easy to mutate and recombine. Pixels would make the genome as big as the image (no compression, no generalisation). - *Is Exercise 1 (ASCII art) implemented?* No — Exercise 1 is design-only in our submission; all the code and experiments are Exercise 2.

Slide p8 — Input / Output / Optimization

Say (~30 s):

> *"The program takes three inputs: the target image, the number of triangles, and the GA hyperparameters — population size, mutation and crossover rates, and which selection/crossover/mutation/survival methods to use. It outputs the best image it found, the full triangle list — position, colour and transparency for each — and performance metrics: fitness, error and how many generations it ran. Internally the optimisation is a genetic algorithm that evolves triangle configurations. Error is turned into a fitness score with* `1 / (1 + k·error)`*, so lower error means higher fitness, and fitness always lands in the interval zero-to-one."*

Points to hit: - Output triangle list is fully self-contained (normalised coords + RGBA + canvas size + background) → the solution can be re-rendered at any resolution without the target. - `score = 1/(1+k·error)`: monotonic decreasing in error, strictly positive, bounded in `(0,1]`.

Likely questions: - *What is* `k`*?* A scaling constant, set to 1 in our runs. It doesn't change the ranking of individuals (the mapping is monotonic); it only shapes how fitness differences are spread out. - *Why map error to fitness at all — why not minimise error directly?* Because several of our selection methods (roulette, universal, Boltzmann) need **strictly positive** fitness values to normalise or exponentiate. `1/(1+k·error)` guarantees that and keeps everything in `(0,1]`. - *What are the GA hyperparameters exactly?* population size, number of generations, optional min-error early stop, selection method (+ tournament size / probability, Boltzmann temperature), crossover method + rate, mutation method + rate, survival strategy, seed.

Slide p10 — Fitness Function (MSE)

Say (~30 s):

> *"The fitness function measures how close a candidate is to the target. We render the candidate's triangles to a canvas the same size as the target, then compute the* ***mean squared error*** *between the two images — pixel by pixel, averaged over the red, green and blue channels, with pixel values scaled to zero-to-one. So MSE itself is between 0 and 1: zero means identical. Lower error, higher fitness, better individual. We chose MSE because it's a simple, cheap, pixel-level similarity measure — and that matters, because fitness gets evaluated for every new individual in every generation, thousands of times per run."*

Points to hit: - The flow on the slide: target + candidate → MSE → error (lower = better) → fitness (higher = better). - MSE is vectorised NumPy — trivial compared to the rendering step, which is the real cost of a fitness evaluation.

Likely questions: - *What's the weakness of MSE?* It's **pixel-wise, not perceptual**. It doesn't see edges, shapes or "recognizability" — two images with the same MSE can look very different. That's exactly why "explore a better perceptual fitness function" is in our future-work slide. Alternatives: SSIM, multi-scale or edge-aware metrics. - *Is fitness recomputed every generation?* No — it's **cached on the individual** and invalidated (set to `None`) whenever a mutation changes that individual. Crossover children are brand-new objects, so they're evaluated once. Individuals that survive unchanged through additive survival keep their cached fitness, so we don't re-render the whole surviving population each generation. - *Why is a fitness evaluation expensive if MSE is cheap?* The cost is **rendering** — rasterising and alpha-blending every triangle onto the canvas — not the MSE arithmetic. - *Does MSE double-count anything / how are channels handled?* One mean over all pixels and all 3 channels together, after `/255`. RGB only (target is converted to RGB on load).

Slide p12 — Crossover (left column)

Say (~30 s):

> *"Crossover combines two parents into offspring. The unit we recombine is a **whole triangle** — we never split a triangle's vertices from its colour. We implemented three methods. **One-point**: pick one cut index, the child takes triangles before it from one parent and after it from the other. **Two-point**: pick two cut points and swap the middle segment. **Uniform**: go triangle by triangle and independently take each one from either parent, roughly fifty-fifty. Crossover is applied to a pair with probability 0.9; otherwise the parents pass through unchanged."*

Points to hit: - Gene granularity = one triangle. Position in the list only matters for **draw order**, so positional linkage between genes is weak. - Offspring are new `Individual`s with fitness reset.

Likely questions: - *Which crossover won, and why?* **Uniform** — lowest median MSE (0.023 vs ~0.030) *and* lowest run-to-run variation. Interpretation: because a triangle is a self-contained unit whose effect is mostly local paint, mixing them one-by-one recombines useful triangles from both parents regardless of position. One- and two-point preserve positional blocks that don't mean much here. - *Isn't uniform crossover usually too disruptive?* For bit-strings, yes. Here each "gene" is a coherent object, so swapping whole triangles rarely destroys a half-built structure — and the experiment backs that up (best median *and* tightest spread). - *What if parents have different lengths?* They can't — triangle count is fixed for a run, so all individuals are the same length. - *What happens when crossover doesn't fire (prob 0.1)?* The two parents are **copied** through unchanged (we copy so that the later in-place mutation step doesn't corrupt the surviving parents — that was an actual bug we fixed).

Slide p13 — Genetic Algorithm Flow

Say (~35 s):

> *"The loop: we start from a randomly generated population — every triangle's vertices and colour drawn uniformly at random. We evaluate fitness. Then each generation: check the termination condition; if not met, **select** parents, **cross** them over, **mutate** the offspring, evaluate the offspring's fitness, and run **survival** to pick the next generation. Repeat. When termination is met — a maximum number of generations, or an error threshold — we return the best individual found."*

Points to hit: - Selection picks `population_size` parents → paired into `population_size` offspring → additive survival merges parents + offspring (2× population) and keeps the best `population_size`. So it's effectively **(μ + λ) with λ = μ**. - Best-so-far MSE is **monotonic under additive survival** (the best can never be dropped); under exclusive survival it isn't.

Likely questions: - *Where is elitism?* Two places in the baseline: "Elite" parent selection, and — more importantly — **additive (μ+λ) survival**, where the current best always competes for a slot and therefore is never lost. - *Is the initial population purely random?* Yes — `n` triangles, each with 3 uniform-random points in `[0,1]²` and a uniform-random RGBA. No seeding from the target. - *Do you re-evaluate the whole population every generation?* Only what changed: new offspring. Cached fitness on unchanged survivors is reused. - *Termination conditions in code?* `generation >= n_generations`, or (if a `min_error` is configured) the last recorded best error `<= min_error`. No stagnation/diversity-based stop. - *Order of mutate vs. evaluate?* Mutate offspring first, then evaluate, then survival — so survival always compares up-to-date fitness.

Slide p15 — Design Choices & Justification (your two rows)

Fitness — MSE

Say (~20 s):

> *"MSE compares the generated image with the target pixel by pixel. It's simple and fast to calculate, and that's a deliberate choice: fitness is evaluated for every offspring in every generation, so the metric has to be cheap. The trade-off is that it's a pixel-level measure, not a perceptual one — which is a limitation we call out in future work."*

Termination

Say (~20 s):

> *"Two termination criteria. **Maximum generations** gives us a hard, predictable limit on computation — important for running fair comparisons across configurations. **Minimum error** lets a run stop early once the result is already good enough, so we don't waste generations. In the experiments we used a fixed generation budget so every configuration gets the same number of iterations."*

Likely questions: - *Is a fixed generation budget a fair comparison?* It's fair on **iterations**, not on **compute** — a larger population or more triangles makes each generation more expensive. We're explicit about that in the analysis; that's why we also report runtime, not just final MSE. - *Why not terminate on convergence/diversity?* Simplicity and comparability. A fixed budget makes all runs directly comparable; the min-error stop covers the "acceptable solution reached" case the assignment asks for. - *Did any run actually hit min-error?* No — the experiment campaign used only the generation cap (`min_error = null`); min-error is available via the CLI.

Slide p27 — Possible improvements (your list)

Say (~30 s):

> *"Four directions. First, **more random seeds** — we used five per configuration, which is enough to see trends but not for strong statistical claims. Second, **a better fitness function** — something that measures visual similarity the way a person would, not just pixel error. Third, **adaptive mutation rates** — start with more exploration and anneal down as the population converges, instead of a fixed rate. And fourth, for **high-resolution images**, split the image into regions and optimise each region separately — the picture on the right was done that way, region by region, which keeps the search tractable as resolution grows."*

Points to hit: - These follow directly from limitations we observed: small seed count → noisy stability numbers; MSE → non-perceptual; fixed mutation rate → same rate means different things per operator and never adapts; single global canvas → search space grows with resolution.

Likely questions: - *Why does region-based help?* A fixed triangle budget spread over a big image gives poor local detail, and the search space grows with the number of triangles. Optimising sub-regions independently keeps each sub-problem small and parallelisable. - *What would adaptive mutation look like concretely?* e.g. tie the rate to generation number or to population diversity / fitness plateau; or self-adaptive rates carried in the genome. - *Why only 5 seeds?* Runtime. A full campaign is 17 configurations; at 5 seeds that's 85 runs of 1000 generations. More seeds was the first thing we'd add with more compute.

General implementation Q&A (cross-cutting — be ready for these)

Why normalised `[0,1]` coordinates and colours? Decouples the genome from canvas resolution — the same individual renders at any size — and puts every gene parameter on the same scale, so mutation (drawing a fresh uniform `[0,1]` value) is meaningful for both a vertex coordinate and a colour channel.

How exactly does a mutation change a triangle? `mutate_triangle` changes **exactly one** of the triangle's 10 parameters: it flips a coin between "a vertex" and "the colour"; if vertex, it picks one of 3 vertices and one of its 2 coordinates and replaces it with a fresh uniform `[0,1]`; if colour, it picks one of the 4 RGBA channels and replaces it. So one mutated triangle = one component resampled.

Then what's the difference between the four mutation operators? (this is the biggest single result in the deck) - **Gene**: with probability `mutation_rate` (0.05), mutate exactly **one** triangle in the individual. → 95% of offspring get *no* mutation at all → diversity collapses → premature convergence. Median MSE ≈ 0.030. - **Multigene (limited)**: pick a random subset of size `M` (`M` uniform in `[0,n)`), then mutate each of those with probability 0.05. Applied every generation, no outer gate → steady trickle of variation. Median MSE ≈ 0.005 — **best**. - **Uniform**: test *every* triangle independently at probability 0.05 (~2–3 mutations per individual per generation). Median MSE ≈ 0.006 — essentially tied with multigene. - **Complete**: with probability 0.05, re-randomise **every** triangle in the individual. Huge disruptive jumps that destroy good solutions → plateaus early. Median MSE ≈ 0.047 — **worst**. - Punchline: the mutation operator had the **largest observed effect on quality**, with almost no runtime difference. Gene and Complete are the two extremes (too little / too much change); Multigene and Uniform sit in the useful middle.

The mutation rate is 0.05 for all four — is that a fair comparison? It's the same *rate*, but the rate *means* something different per operator (probability of one mutation vs. per-triangle probability vs. probability of a full reset). So this compares the operators *as we'd actually use them at a common rate*, not at equal expected mutation counts. We state that explicitly in the analysis.

Why did 20 triangles beat 50 and 100? (counter-intuitive result) The budget is fixed at 1000 *generations*, not fixed compute. More triangles = a higher-dimensional search space (10 params each) → slower progress per generation, and each generation is also slower to render. The Argentina flag is basically three flat colour bands plus a small sun, so ~20 triangles already have enough representational capacity; the extra triangles are just more parameters to optimise and more opportunities to add noise. Under a much larger budget the ranking could flip.

Why did a bigger population help when more triangles didn't? Population size adds *parallel exploration* and a stronger selection signal every generation — it directly improves per-generation improvement, at roughly linear runtime cost (pop 200 ≈ 4× the runtime of pop 50, and ≈ 50% lower MSE). Triangle count adds *representational capacity*, which isn't the bottleneck for this target, and enlarges the search space.

Additive vs. exclusive survival — why is additive better? Additive `(μ+λ)` lets good parents survive if offspring don't beat them, so quality is monotonic and never regresses (MSE ≈ 0.030). Exclusive `(μ,λ)` throws all parents away every generation, so a bad batch of offspring loses hard-won quality (MSE ≈ 0.053). Exclusive needs `#offspring ≥ population_size` (there's an assert); with λ=μ that holds exactly.

How is rendering done, and why does draw order matter? pygame `Surface`, filled white. Each triangle is drawn onto its own transparent `SRCALPHA` layer, then alpha-blended onto the canvas with `blit`, **in list order**. Later triangles paint over earlier ones, so the order of triangles in the genome is part of the phenotype — the JSON export preserves it.

Is the run deterministic? Yes, given a seed — a single `numpy` `default_rng(seed)` drives everything (and Python's `random` is seeded too). Same seed + same config + same target bytes → same result. The experiment runner even hashes the target image and the full effective config to detect drift.

How are experiments made reproducible / not silently reused? Each run's directory name encodes selection/crossover/mutation/triangles/population/generations + a SHA-256 of *all* effective settings and the target bytes. Before reusing a cached result the runner re-validates the whole config, the target hash, the completed generation budget, the histories, both timing measurements, and the exported files. Mismatches raise an error instead of being reused.

Two timing numbers — what's the difference? `ga_elapsed_s` = wall time around `run_ga` only (init + evolution + in-loop metrics). `elapsed` = the whole child process (Python startup + image load + GA + writing image/plot/JSON). Runtime figures use GA time; total time is kept in the tables.

What's the "final validation" run? We combined the best setting from each *separate* experiment — deterministic tournament, uniform crossover, uniform mutation, additive survival, population 100 — and ran 2000 generations. Result: MSE 0.00104 (fitness 0.99896). It's a **single** run that changes several factors *and* the budget at once, so it's deliberately kept out of the controlled comparison — it's a sanity check that the combined settings work well together, not evidence.

Why does the figure caption say `flag_100.png` but the image is the Argentina flag? `flag_100.png` *is* the Argentina flag, downscaled to 100 px. Earlier pilots used other targets (Munch's *The Scream*); the title slide uses *Starry Night* purely as an illustration. All the numbers in the results section are on the 100-px Argentina flag.